



UNIVERSIDAD DE LAS AMÉRICAS(UDLA)

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

CARRERA DE INGENIERÍA EN CIBERSEGURIDAD

ASIGNATURA:
PROGRAMACIÓN II

CÓDIGO:
ISWZ1103

ACTIVIDAD:
TALLER DE PROGRAMACIÓN 2

TEMA:
IMPLEMENTACIÓN DE CLASES Y OBJETOS EN JAVA

ESTUDIANTES:
MARTIN AIMACAÑA, ALEX PALMA

DOCENTE:
MSC. KEVIN GUZMÁN

FECHA:
25 DE ABRIL DE 2026

OBJETIVO

Aplicar los conocimientos adquiridos sobre clases y objetos en Java mediante la implementación de ejercicios prácticos, utilizando atributos privados, constructores, métodos establecer y obtener, métodos de cálculo y clases de prueba para demostrar el funcionamiento de cada programa.

DESCRIPCIÓN DE LA ACTIVIDAD

En el presente taller se desarrollaron varios ejercicios del capítulo de clases y objetos en Java. Cada ejercicio fue implementado mediante una clase principal con sus respectivos atributos, constructor, métodos get y set, y métodos adicionales según el problema planteado.

Además, se crearon clases de prueba para ingresar datos, crear objetos y mostrar los resultados obtenidos. Para cumplir con las indicaciones de la actividad, se utilizaron diferentes formas de entrada de datos como ventanas emergentes, Scanner y BufferedReader.

HERRAMIENTAS UTILIZADAS

Lenguaje de programación: Java

Entorno de desarrollo: IntelliJ IDEA

Entrada de datos: JOptionPane, Scanner y BufferedReader

Control de versiones: GitHub

Sistema operativo: Windows

EJERCICIO 1: LA CLASE FACTURA

Descripción

En este ejercicio se creó una clase llamada Factura, la cual representa una factura de venta de un artículo en una ferretería. La clase contiene cuatro atributos: número de pieza, descripción de la pieza, cantidad y precio por artículo.

También se implementó un constructor para inicializar los datos, métodos establecer y obtener para cada atributo, y un método llamado obtenerMontoFactura(), encargado de calcular el monto total de la factura multiplicando la cantidad por el precio del artículo.

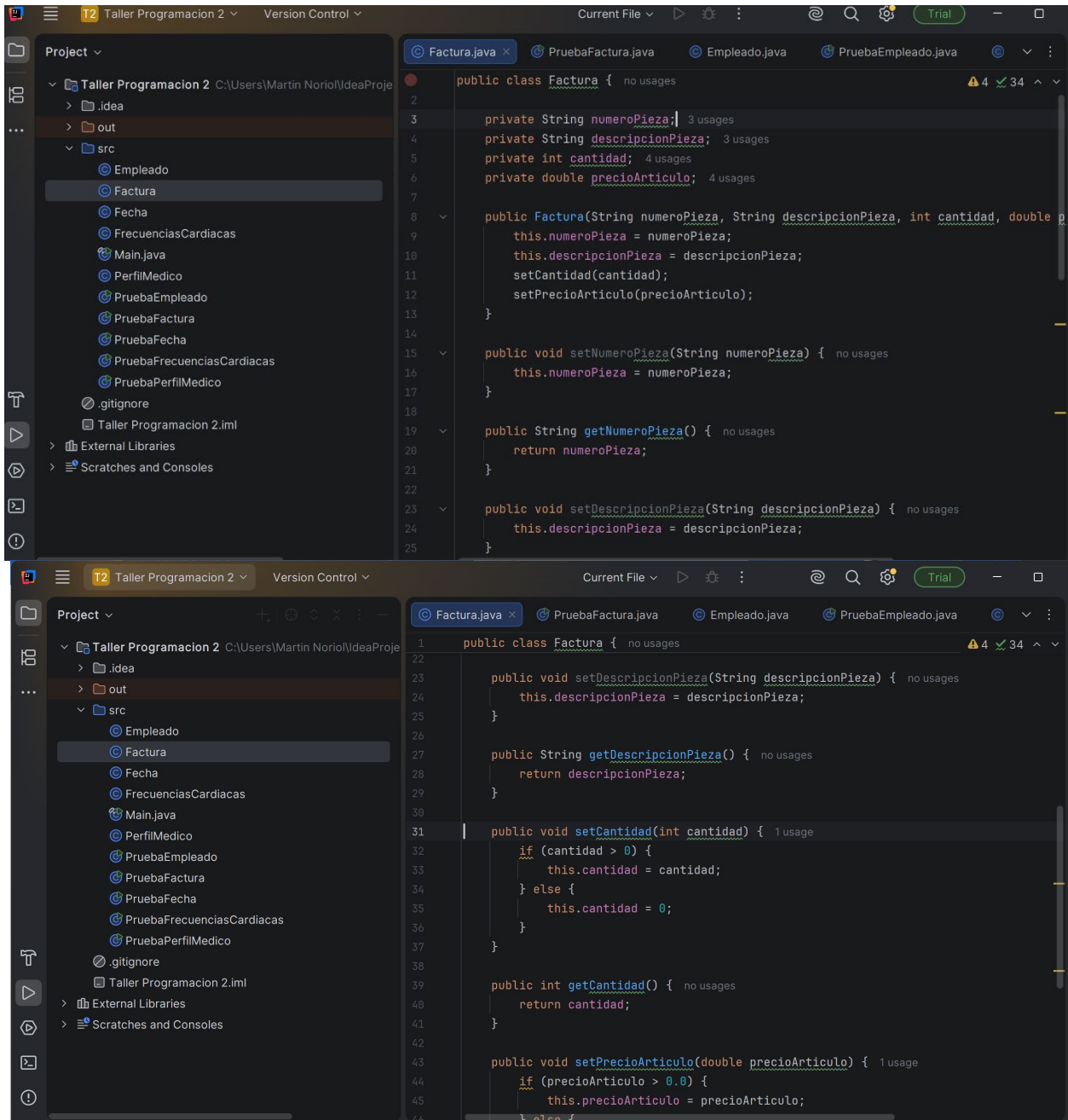
Se aplicaron validaciones para que, si la cantidad ingresada no es positiva, se establezca en 0; y si el precio por artículo no es positivo, se establezca en 0.0.

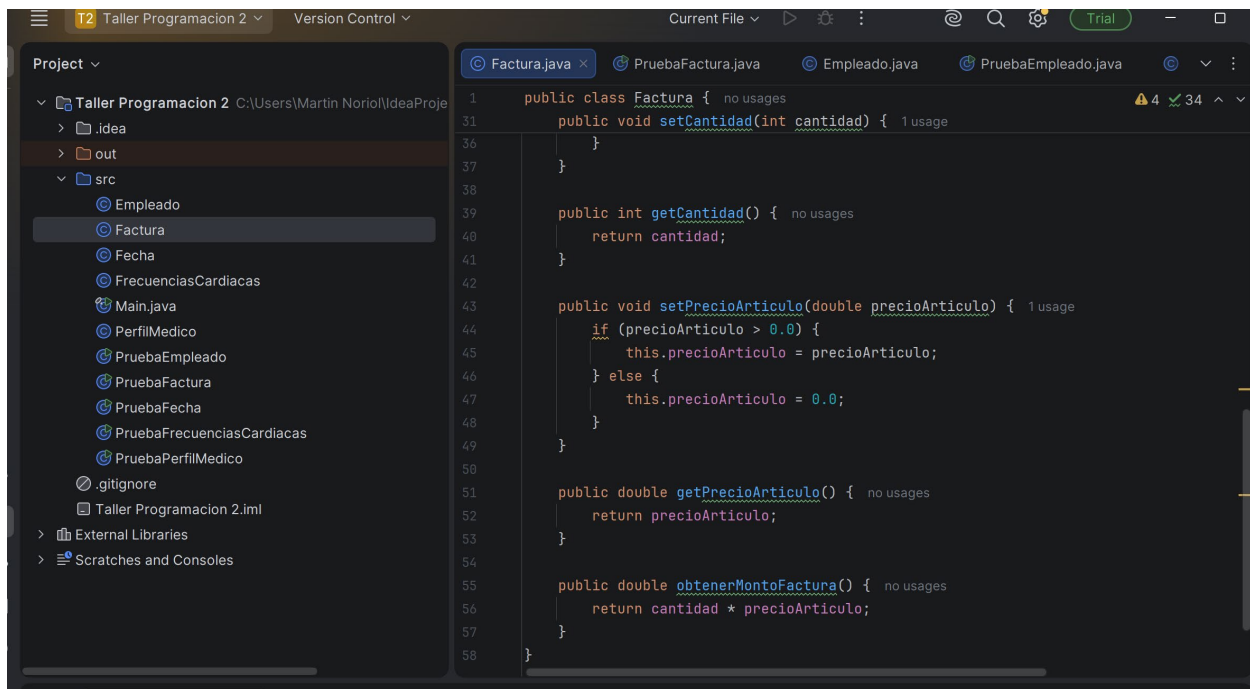
Herramienta de entrada utilizada

Ventanas emergentes mediante JOptionPane.

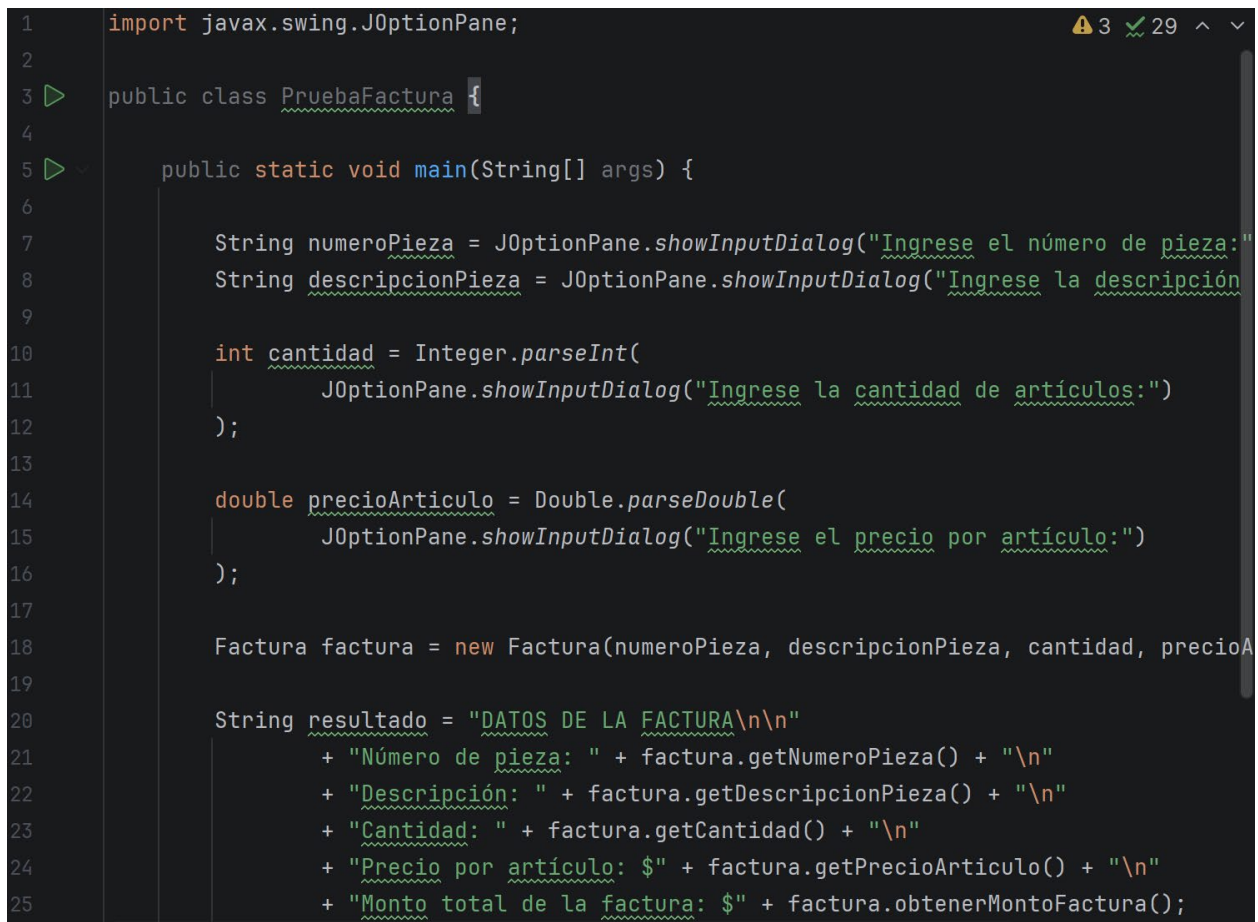
Código fuente

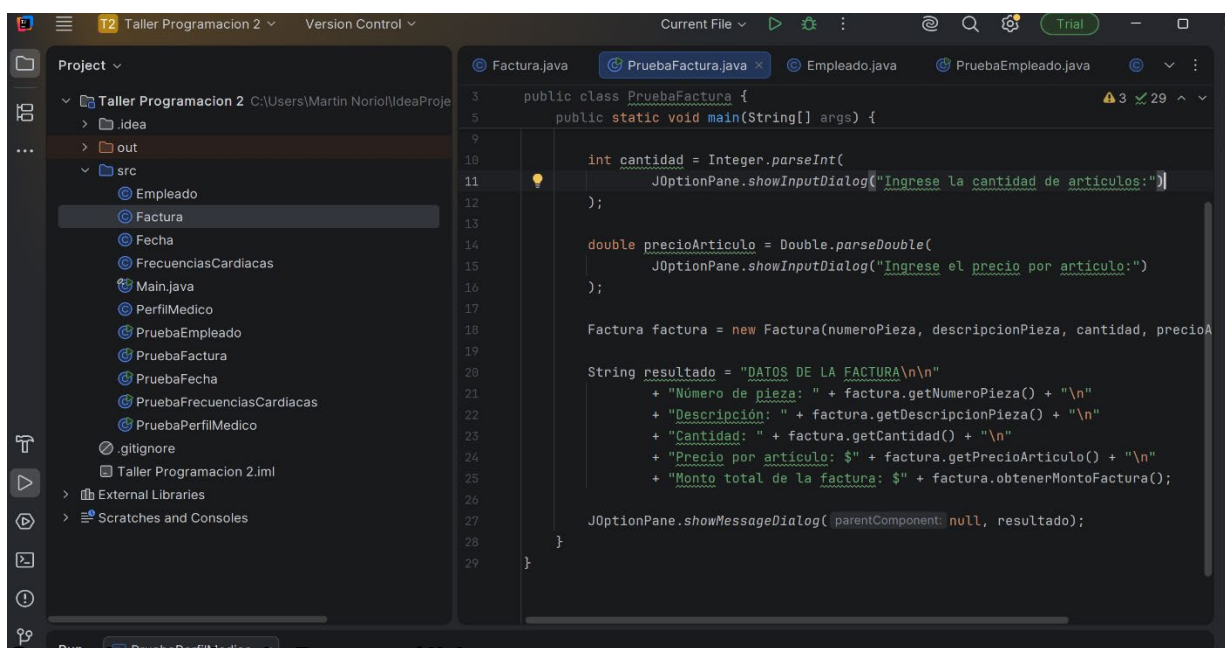
Captura de Factura.java





Captura de PruebaFactura.java





Caso de prueba 1: Datos validos

Número de pieza: b001

Descripción: martillo

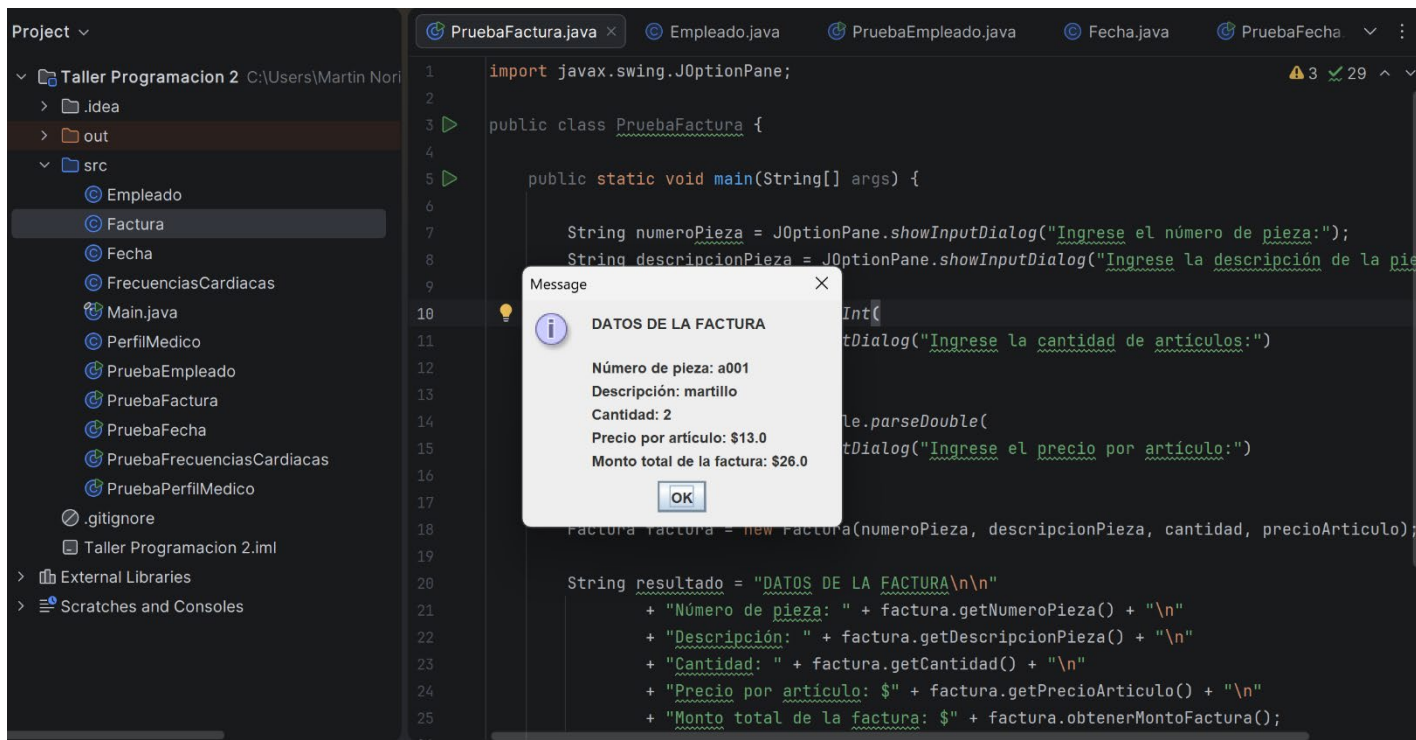
Cantidad: 2

Precio por artículo: 13

Resultado esperado

Monto total de la factura: 26.0

Evidencia de ejecución



Caso de prueba 2: Cantidad negativa

Número de pieza: b002

Descripción: clavos

Cantidad: -5

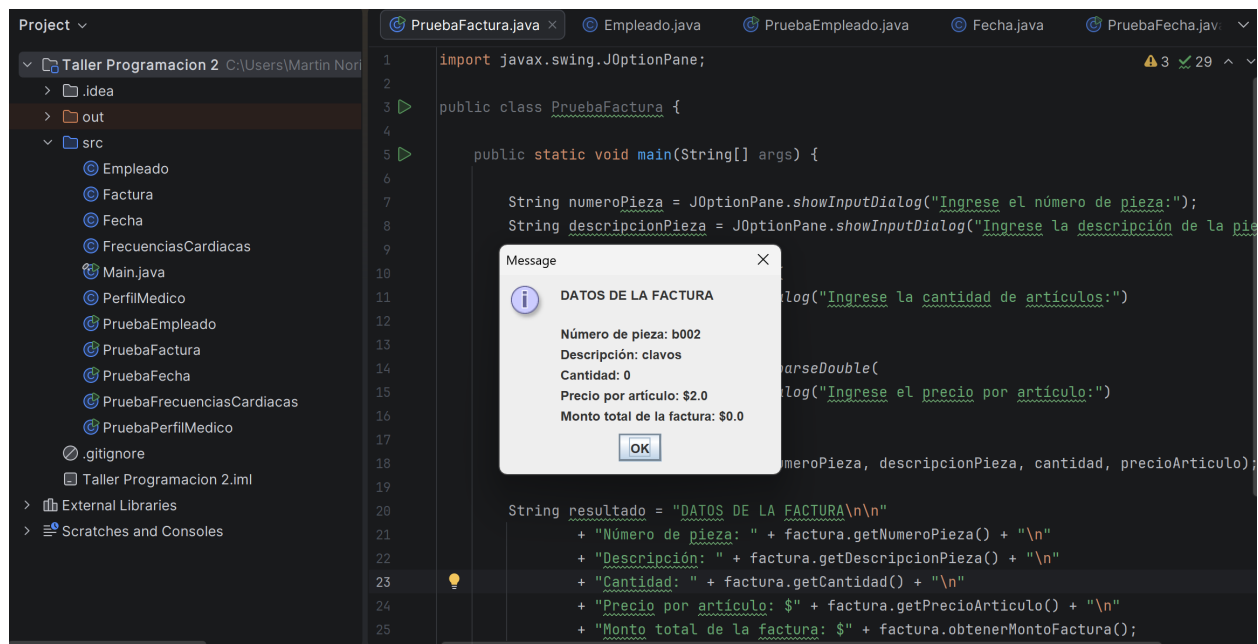
Precio por artículo: 2

Resultado esperado

Cantidad: 0

Monto total de la factura: 0.0

Evidencia de ejecución



Caso de prueba 3: Precio negativo

Número de pieza: c003

Descripción: alicate

Cantidad: 3

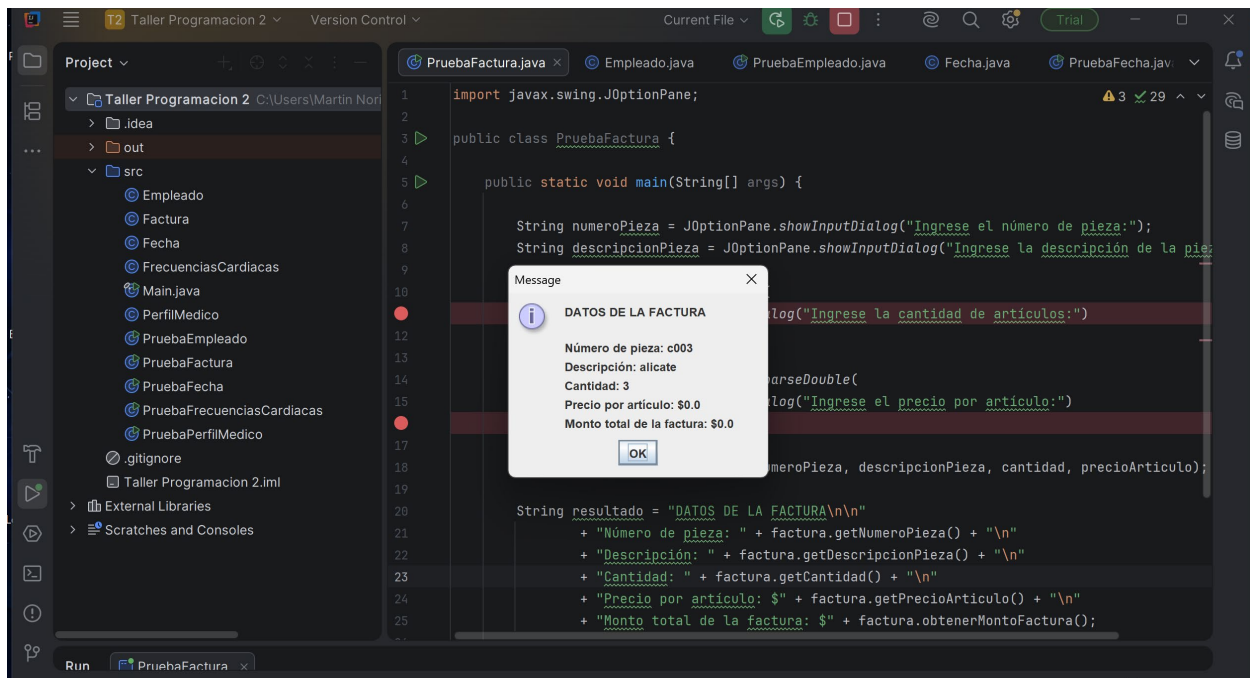
Precio por artículo: -4.5

Resultado esperado

Precio por artículo: 0.0

Monto total de la factura: 0.0

Evidencia de ejecución



EJERCICIO 2: LA CLASE EMPLEADO

Descripción

En este ejercicio se creó una clase llamada Empleado, la cual contiene tres atributos: primer nombre, apellido paterno y salario mensual. La clase incluye un constructor para inicializar los datos y métodos establecer y obtener para cada variable de instancia.

También se implementó un método para calcular el salario anual de cada empleado. Posteriormente, se aplicó un aumento del 10% al salario mensual de cada empleado y se volvió a mostrar el salario anual actualizado.

El programa permite crear dos objetos de tipo Empleado y demostrar el funcionamiento de la clase mediante una clase de prueba llamada PruebaEmpleado.

Herramienta de entrada utilizada

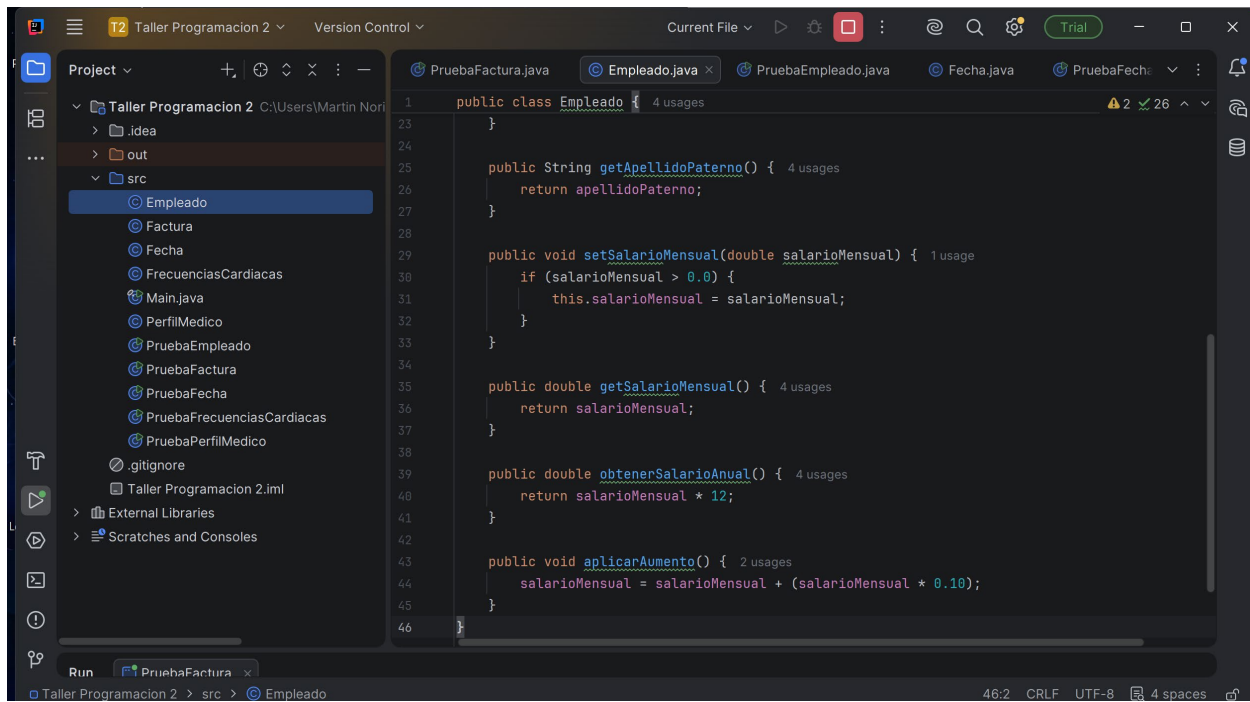
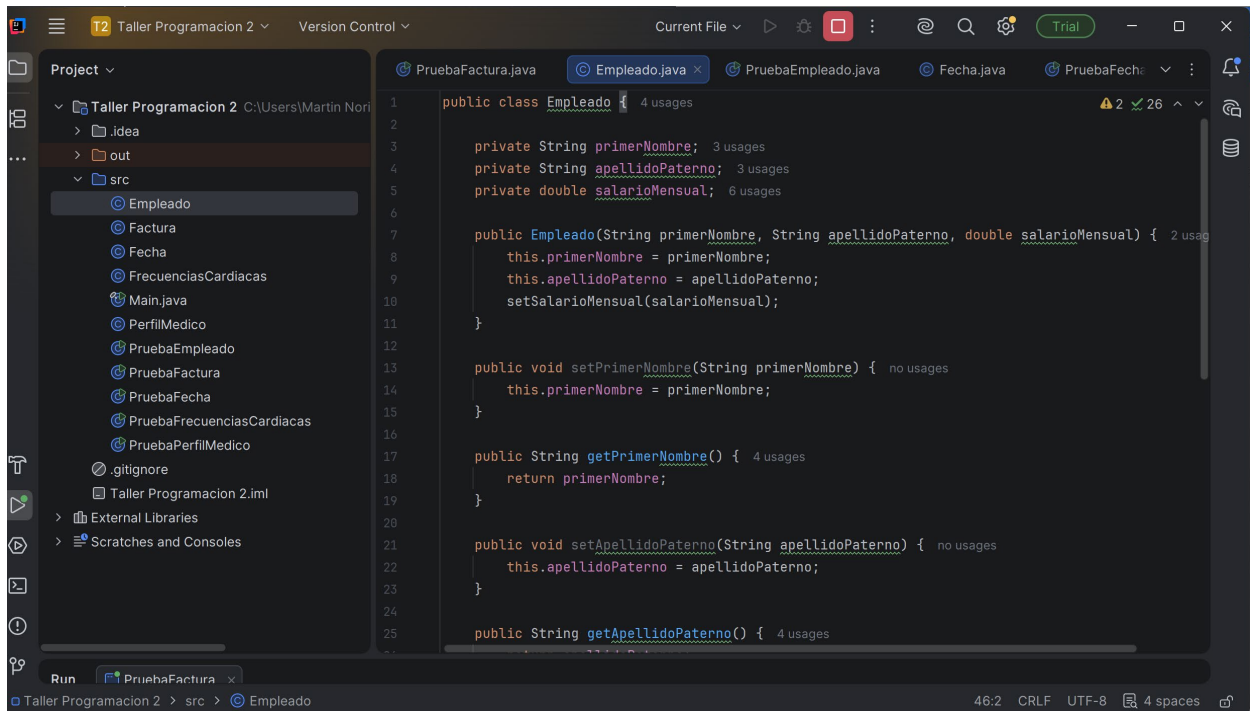
Ventanas emergentes mediante JOptionPane.

O, si al final lo dejaste con Scanner, pon esto:

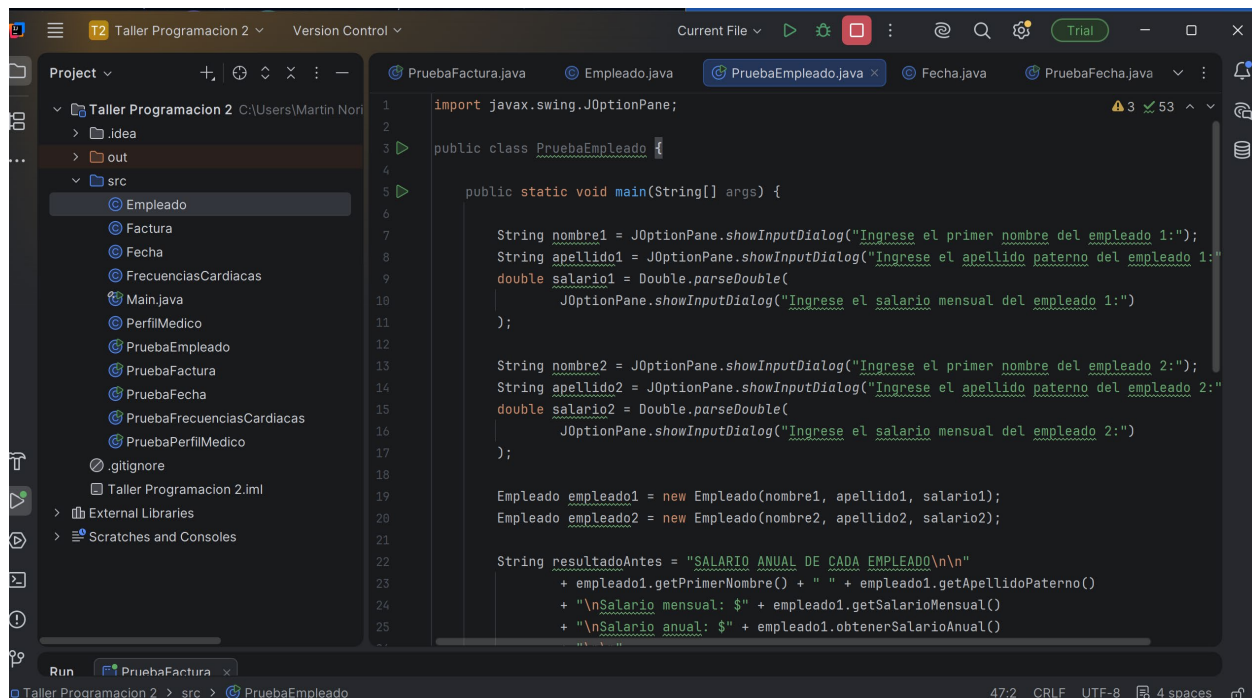
Entrada de datos por consola mediante Scanner.

Código fuente

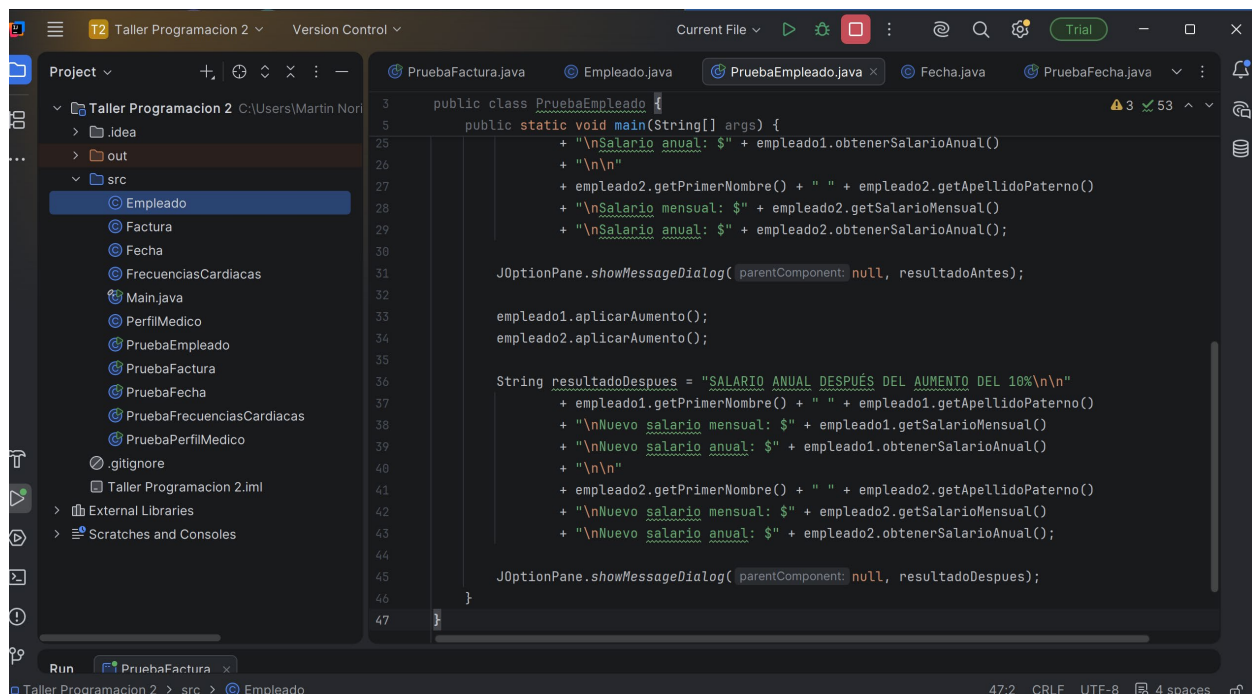
Captura de Empleado.java



Captura de PruebaEmpleado.java



```
1 import javax.swing.JOptionPane;
2
3 public class PruebaEmpleado {
4
5     public static void main(String[] args) {
6
7         String nombre1 = JOptionPane.showInputDialog("Ingrese el primer nombre del empleado 1:");
8         String apellido1 = JOptionPane.showInputDialog("Ingrese el apellido paterno del empleado 1:");
9         double salario1 = Double.parseDouble(
10             JOptionPane.showInputDialog("Ingrese el salario mensual del empleado 1:"));
11     };
12
13     String nombre2 = JOptionPane.showInputDialog("Ingrese el primer nombre del empleado 2:");
14     String apellido2 = JOptionPane.showInputDialog("Ingrese el apellido paterno del empleado 2:");
15     double salario2 = Double.parseDouble(
16         JOptionPane.showInputDialog("Ingrese el salario mensual del empleado 2:"));
17 };
18
19 Empleado empleado1 = new Empleado(nombre1, apellido1, salario1);
20 Empleado empleado2 = new Empleado(nombre2, apellido2, salario2);
21
22 String resultadoAntes = "SALARIO ANUAL DE CADA EMPLEADO\n\n"
23     + empleado1.getPrimerNombre() + " " + empleado1.getApellidoPaterno()
24     + "\nSalario mensual: $" + empleado1.getSalarioMensual()
25     + "\nSalario anual: $" + empleado1.obtenerSalarioAnual();
```



```
25     + "\nSalario anual: $" + empleado1.obtenerSalarioAnual()
26     + "\n\n"
27     + empleado2.getPrimerNombre() + " " + empleado2.getApellidoPaterno()
28     + "\nSalario mensual: $" + empleado2.getSalarioMensual()
29     + "\nSalario anual: $" + empleado2.obtenerSalarioAnual();
30
31 JOptionPane.showMessageDialog(parentComponent: null, resultadoAntes);
32
33 empleado1.aplicarAumento();
34 empleado2.aplicarAumento();
35
36 String resultadoDespues = "SALARIO ANUAL DESPUÉS DEL AUMENTO DEL 10%\n\n"
37     + empleado1.getPrimerNombre() + " " + empleado1.getApellidoPaterno()
38     + "\nNuevo salario mensual: $" + empleado1.getSalarioMensual()
39     + "\nNuevo salario anual: $" + empleado1.obtenerSalarioAnual()
40     + "\n\n"
41     + empleado2.getPrimerNombre() + " " + empleado2.getApellidoPaterno()
42     + "\nNuevo salario mensual: $" + empleado2.getSalarioMensual()
43     + "\nNuevo salario anual: $" + empleado2.obtenerSalarioAnual();
44
45 JOptionPane.showMessageDialog(parentComponent: null, resultadoDespues);
46
47 }
```

Caso de prueba

Empleado 1:
Nombre: Carlos

Apellido: Pérez
Salario mensual: 500

Empleado 2:
Nombre: Ana
Apellido: Gómez
Salario mensual: 700

Resultado esperado antes del aumento

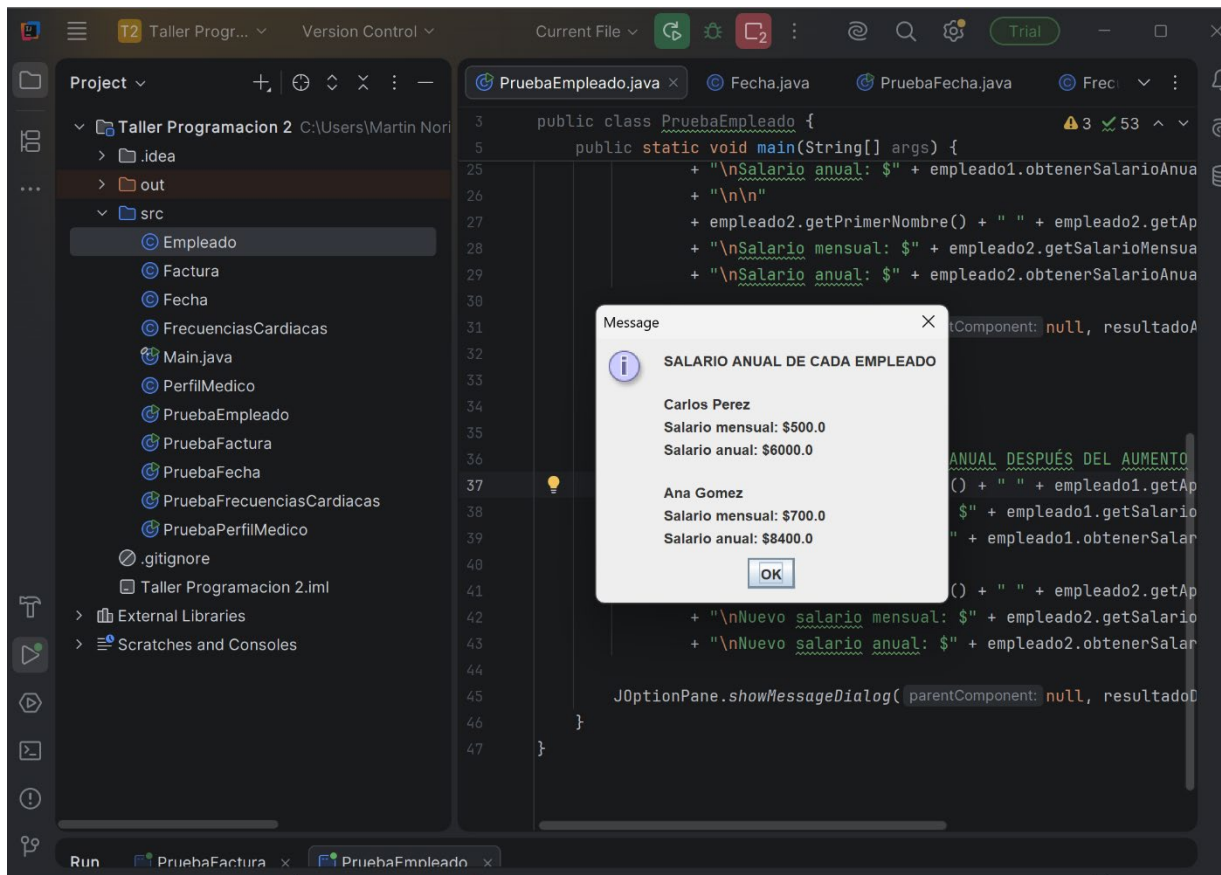
Carlos Pérez tiene un salario anual de: 6000.0
Ana Gómez tiene un salario anual de: 8400.0

Resultado esperado después del aumento del 10%

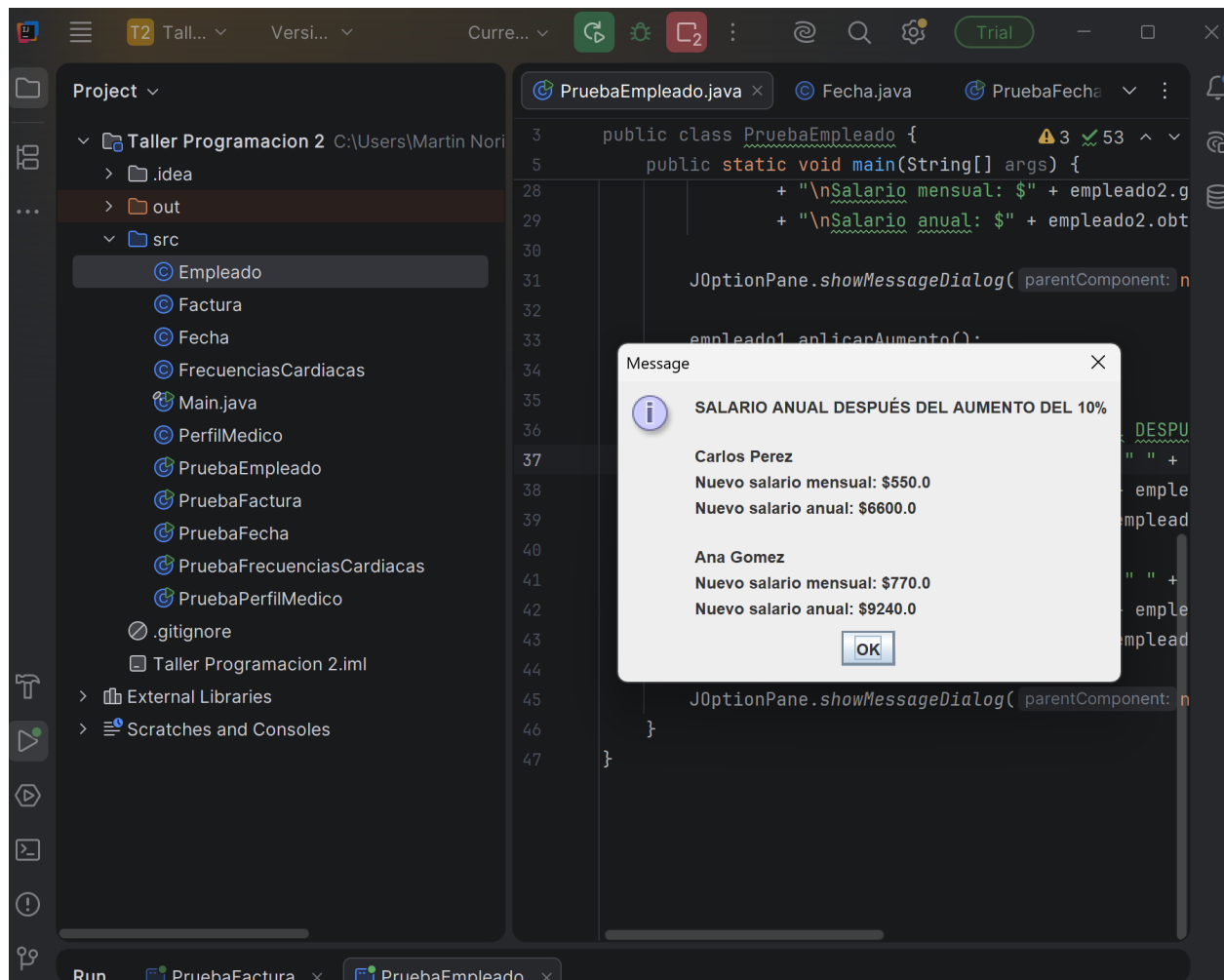
Carlos Pérez tiene un salario anual de: 6600.0
Ana Gómez tiene un salario anual de: 9240.0

Evidencia de ejecución

Captura del salario anual antes del aumento



Captura del salario anual después del aumento del 10%



EJERCICIO 3: LA CLASE FECHA

Descripción

En este ejercicio se creó una clase llamada Fecha, la cual incluye tres variables de instancia: mes, día y año. La clase tiene un constructor que inicializa los tres atributos y métodos establecer y obtener para cada variable.

Además, se implementó el método mostrarFecha(), que permite mostrar la fecha en formato mes/día/año, separada por barras diagonales.

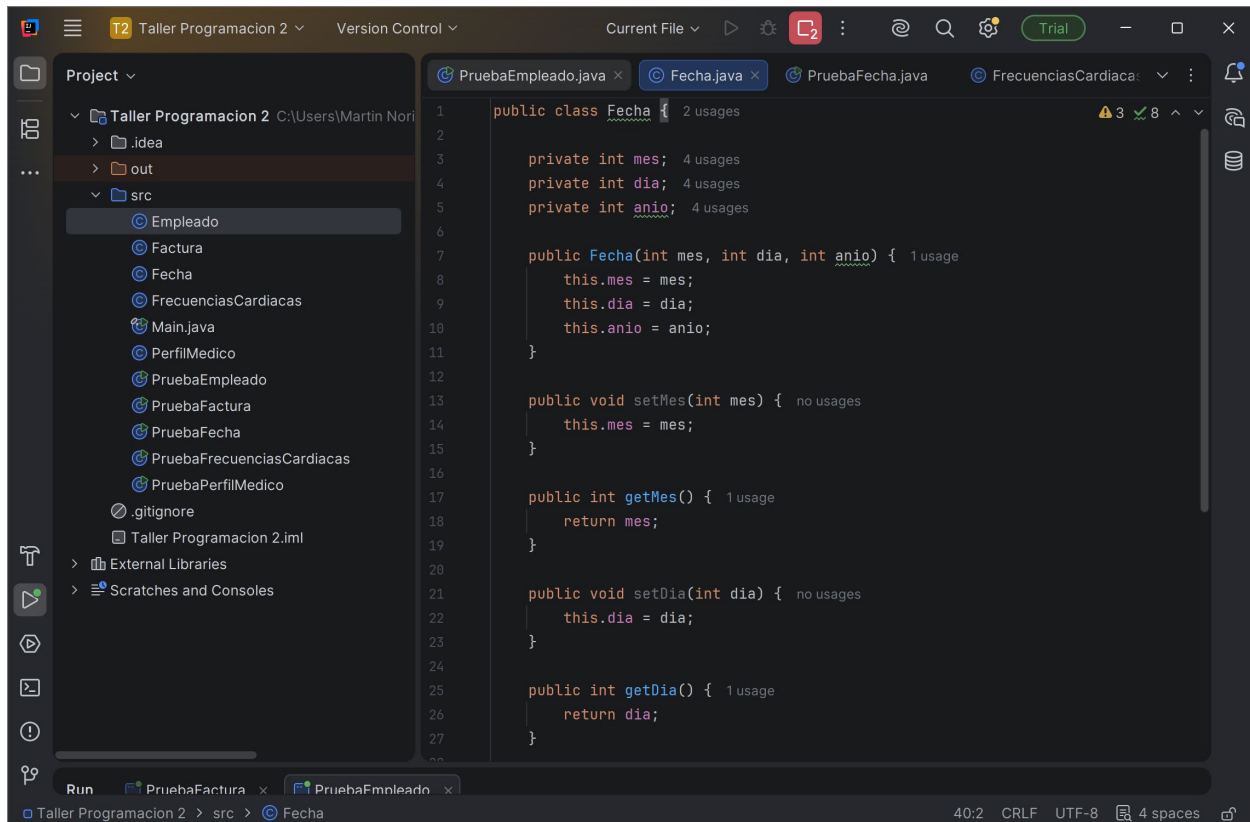
Según el enunciado, se asume que los valores ingresados son correctos, por lo tanto, no fue necesario realizar validaciones adicionales para el mes, día o año.

Herramienta de entrada utilizada

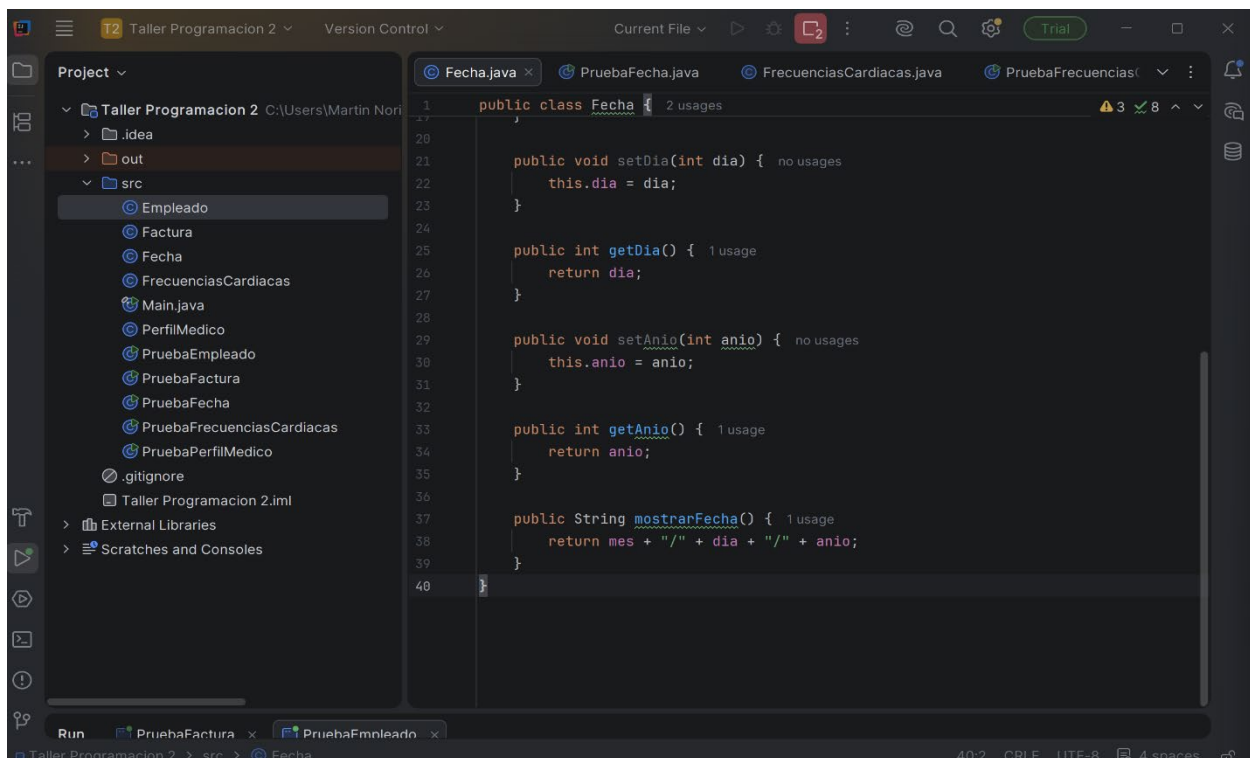
Ventanas emergentes mediante JOptionPane.

Código fuente

Captura de Fecha.java



Captura de PruebaFecha.java



Caso de prueba

Mes: 10

Día: 25

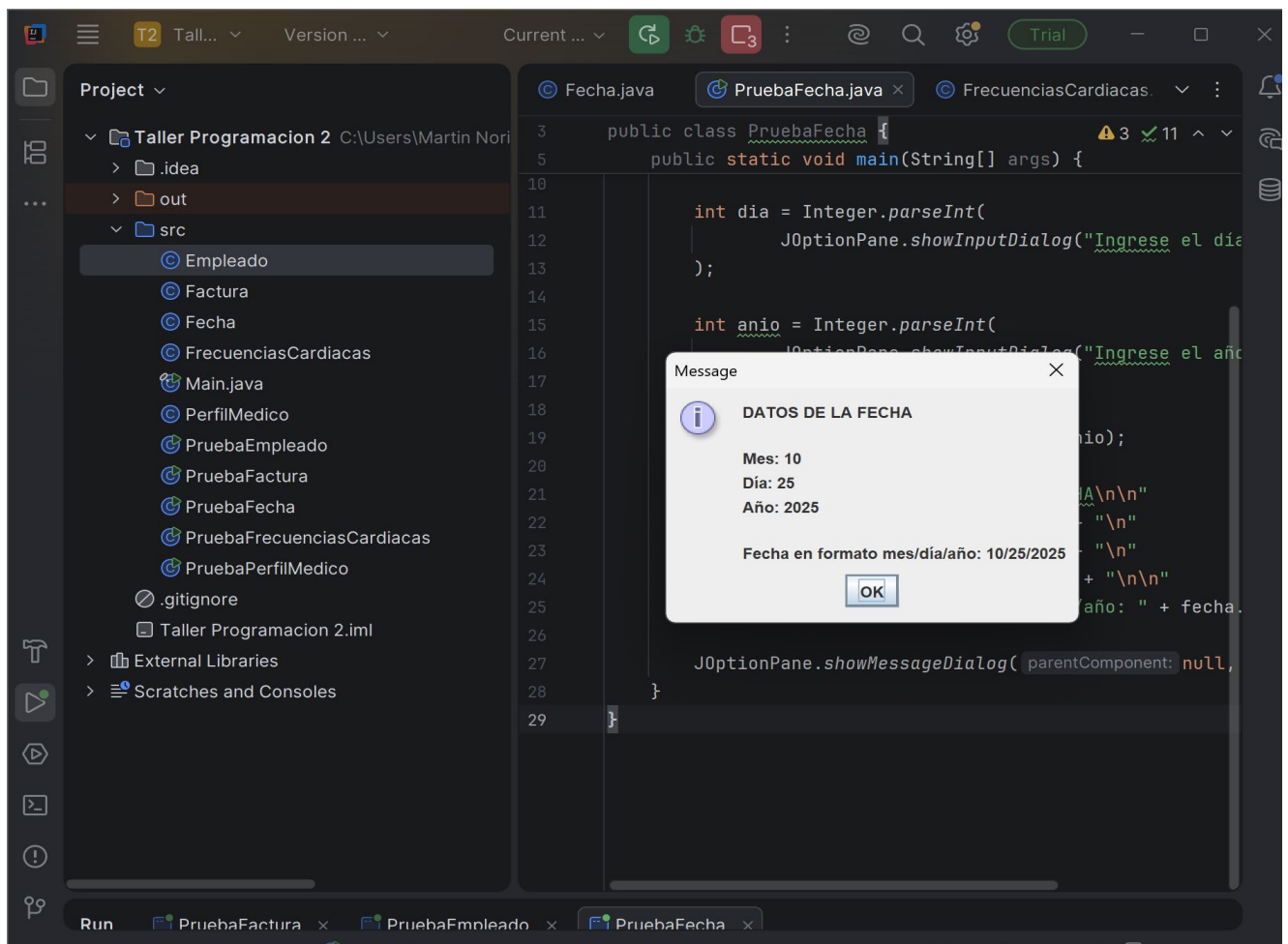
Año: 2025

Resultado esperado

Fecha en formato mes/día/año: 10/25/2025

Evidencia de ejecución

Captura del resultado final



EJERCICIO 4: CALCULADORA DE FRECUENCIA CARDÍACA ESPERADA

Descripción

En este ejercicio se creó una clase llamada FrecuenciasCardiacas, la cual permite calcular información relacionada con la frecuencia cardíaca de una persona. La clase contiene atributos como primer nombre, apellido y fecha de nacimiento, separada en mes, día y año.

Se implementaron métodos para calcular la edad de la persona, la frecuencia cardíaca máxima y el rango de frecuencia cardíaca esperada, correspondiente al 50% y 85% de la frecuencia máxima.

La frecuencia cardíaca máxima se calcula mediante la fórmula:

Frecuencia cardíaca máxima = 220 - edad

El rango esperado se calcula así:

Frecuencia mínima esperada = frecuencia máxima * 0.50

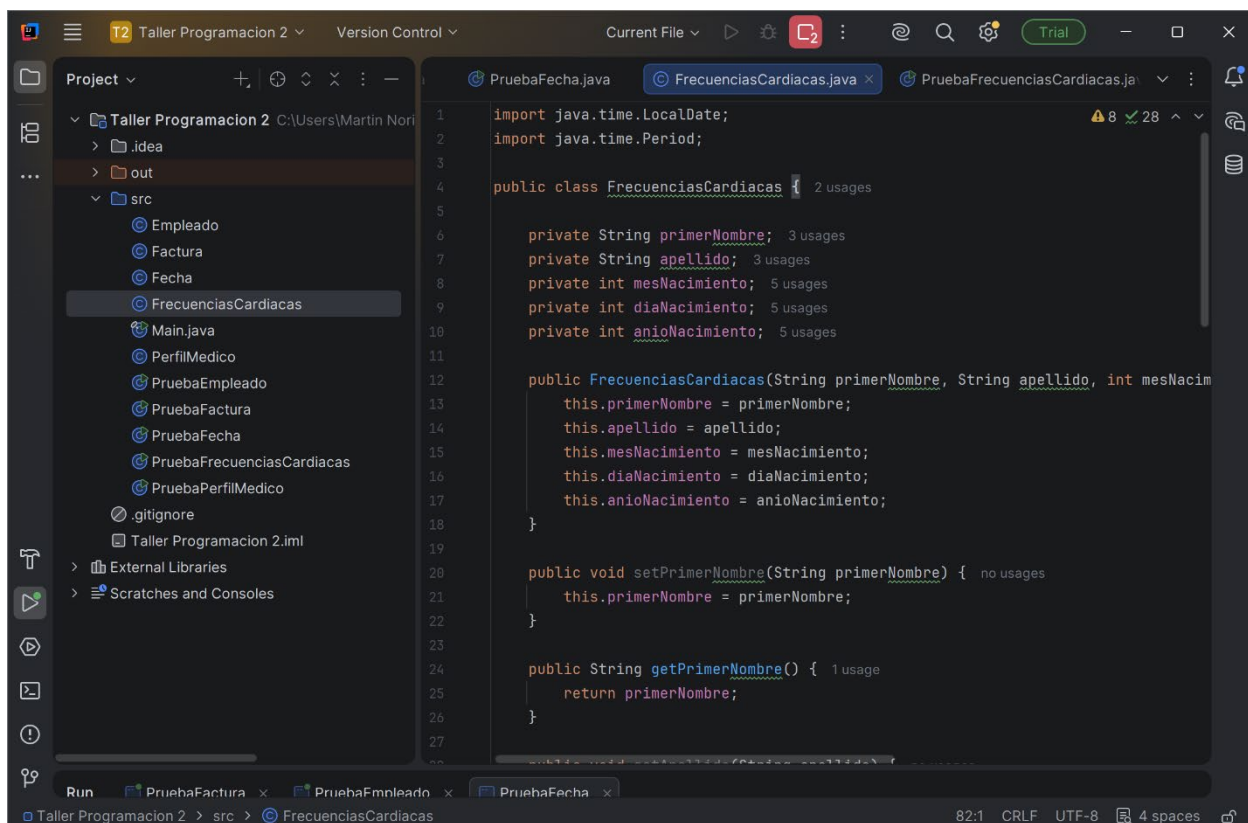
Frecuencia máxima esperada = frecuencia máxima * 0.85

Herramienta de entrada utilizada

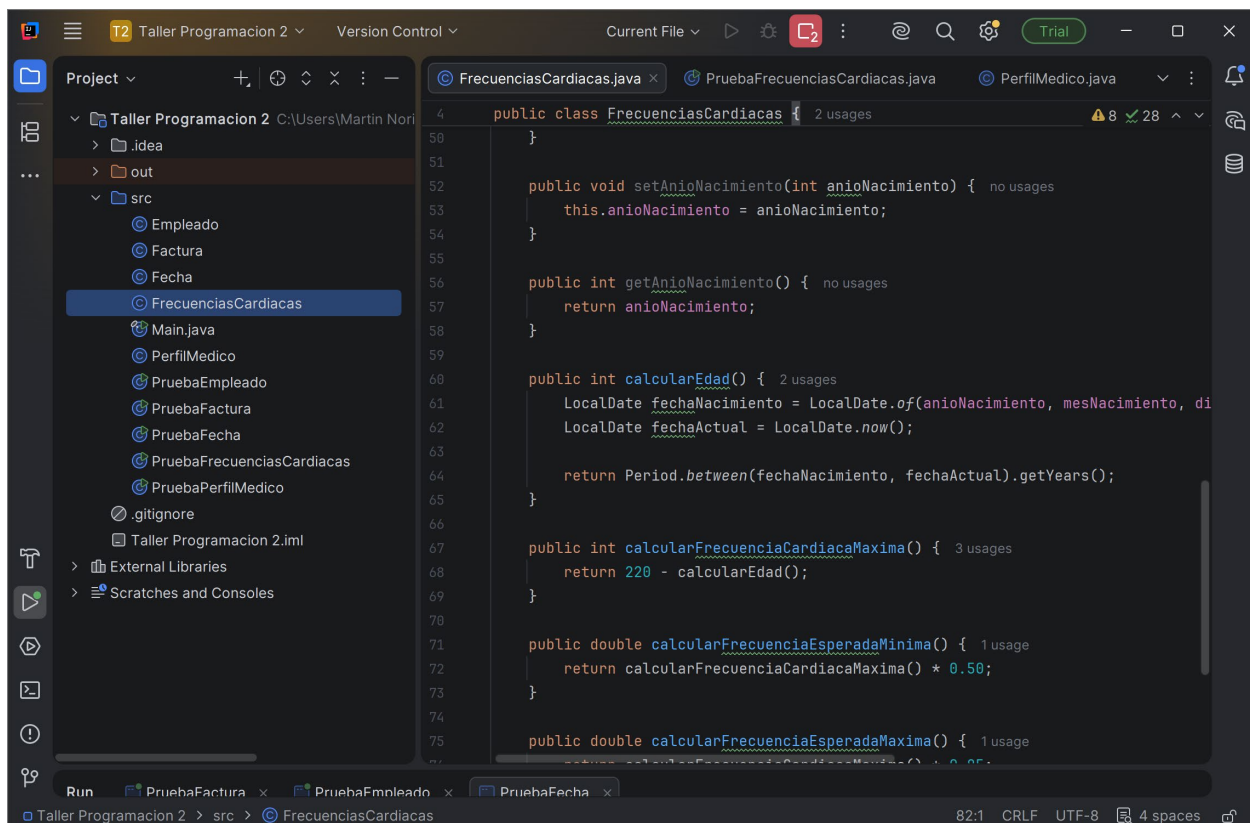
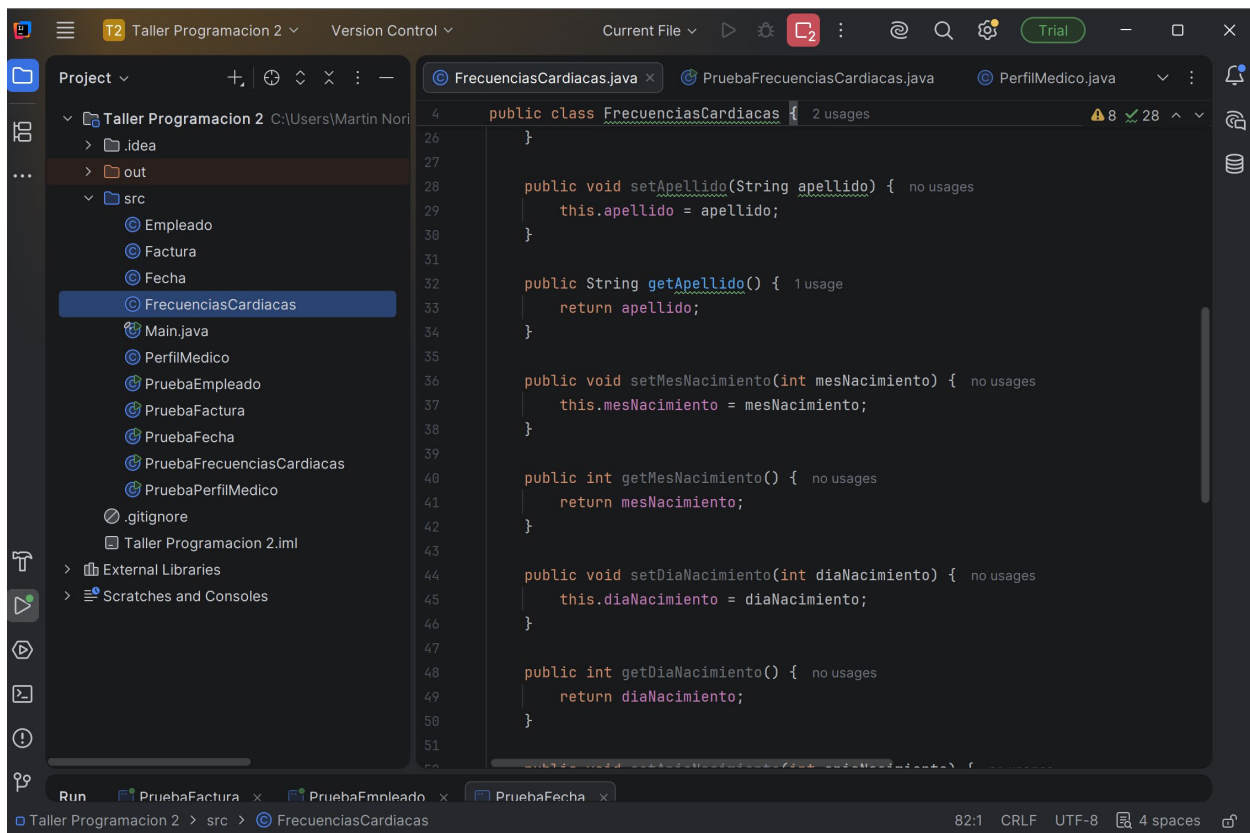
Entrada de datos por consola mediante Scanner.

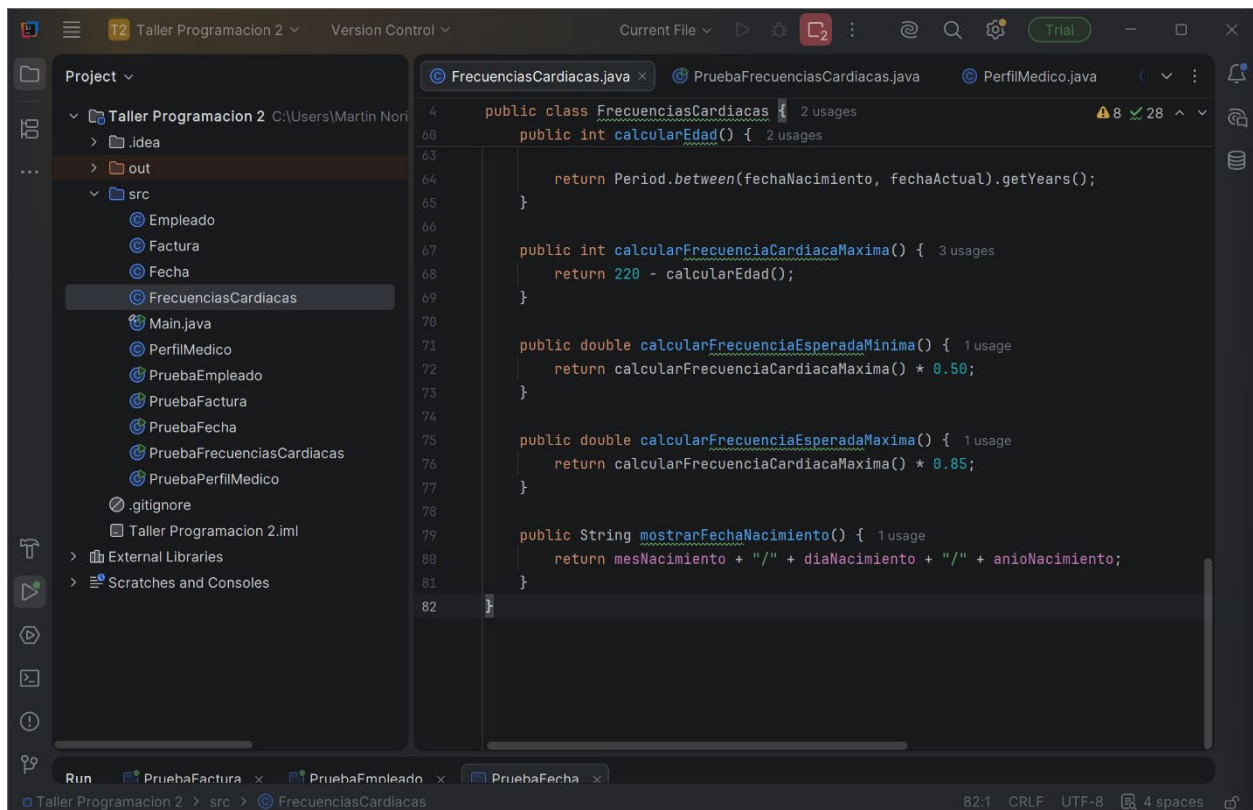
Código fuente

Captura de FrecuenciasCardiacas.java

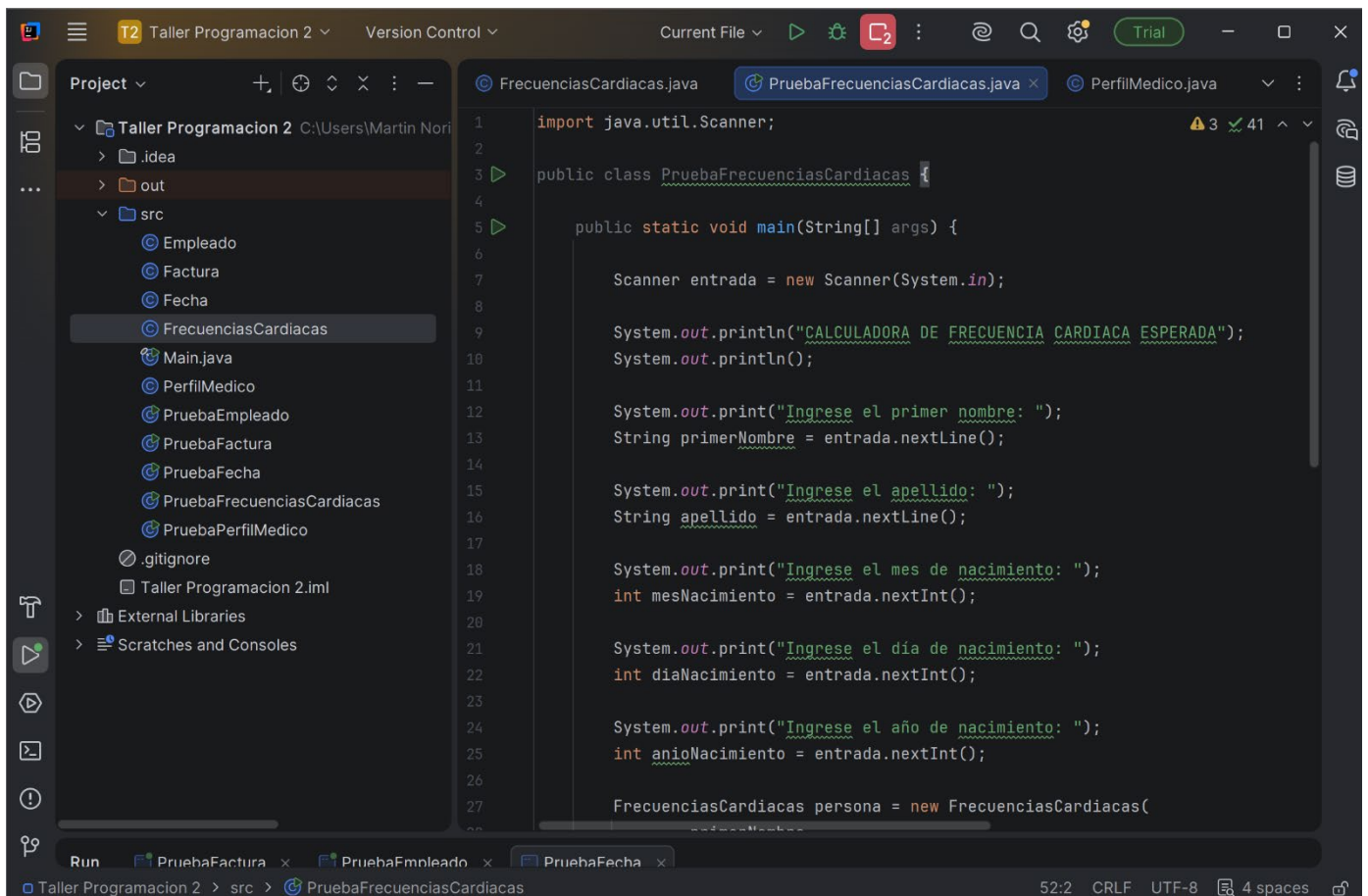


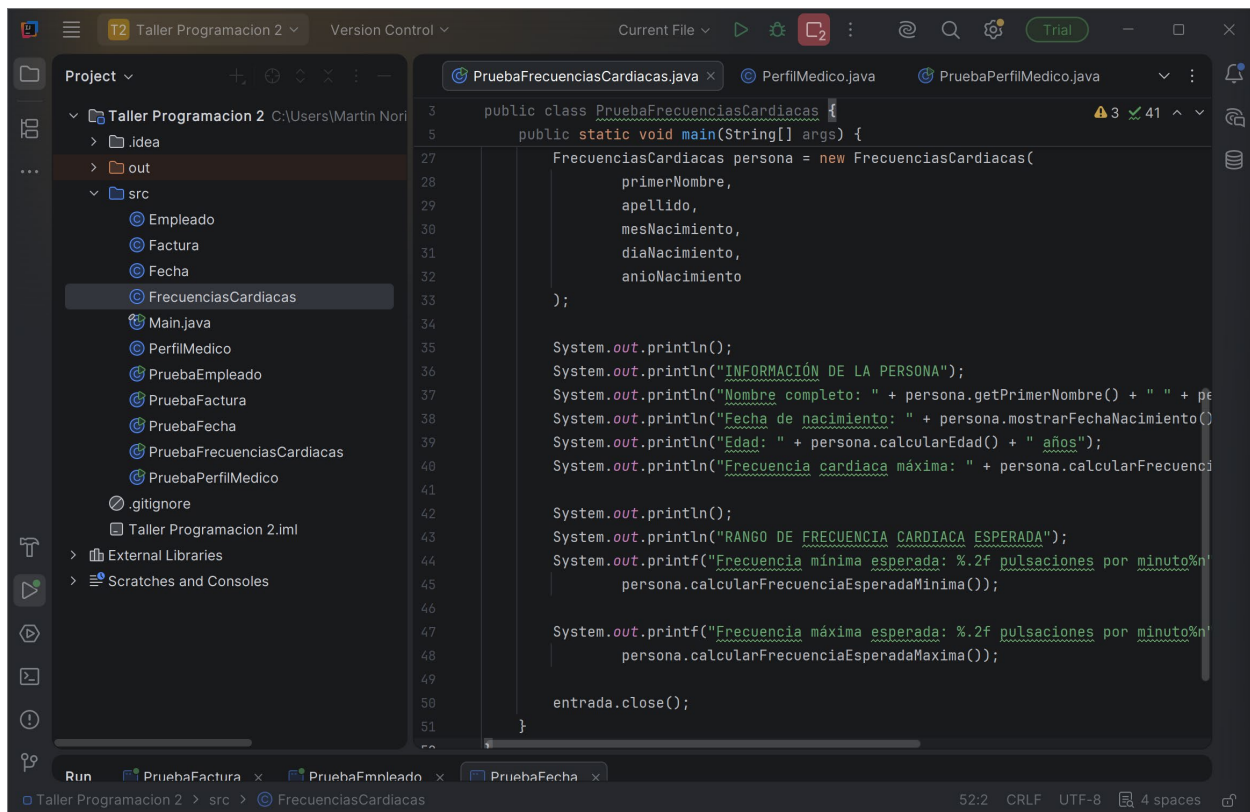
```
1 import java.time.LocalDate;
2 import java.time.Period;
3
4 public class FrecuenciasCardiacas {
5
6     private String primerNombre;
7     private String apellido;
8     private int mesNacimiento;
9     private int diaNacimiento;
10    private int anioNacimiento;
11
12    public FrecuenciasCardiacas(String primerNombre, String apellido, int mesNacim
13        this.primerNombre = primerNombre;
14        this.apellido = apellido;
15        this.mesNacimiento = mesNacimiento;
16        this.diaNacimiento = diaNacimiento;
17        this.anioNacimiento = anioNacimiento;
18    }
19
20    public void setPrimerNombre(String primerNombre) {
21        this.primerNombre = primerNombre;
22    }
23
24    public String getPrimerNombre() {
25        return primerNombre;
26    }
27
28 }
```



Captura de PruebaFrecuenciasCardiacas.java





Caso de prueba

Primer nombre: Carlos

Apellido: Pérez

Mes de nacimiento: 5

Día de nacimiento: 10

Año de nacimiento: 2005

Resultado esperado

Edad: depende de la fecha actual

Frecuencia cardíaca máxima: 220 - edad

Rango esperado mínimo: 50% de la frecuencia máxima

Rango esperado máximo: 85% de la frecuencia máxima

Evidencia de ejecución

Captura de los datos ingresados

```
"C:\Users\Martin Norio1\.jdk\openjdk-26.0.1\bin\j
CALCULADORA DE FRECUENCIA CARDIACA ESPERADA

Ingrese el primer nombre: Carlos
Ingrese el apellido: Perez
Ingrese el mes de nacimiento: 5
Ingrese el día de nacimiento: 10
Ingrese el año de nacimiento: 2005
```

Captura del resultado con edad, frecuencia cardíaca máxima y rango esperado

```
Run PruebaFactura x PruebaEmpleado x PruebaFrecuenciasCardiac
INFORMACIÓN DE LA PERSONA
Nombre completo: Carlos Perez
Fecha de nacimiento: 5/10/2005
Edad: 20 años
Frecuencia cardiaca máxima: 200 pulsaciones por minuto

RANGO DE FRECUENCIA CARDIACA ESPERADA
Frecuencia mínima esperada: 100,00 pulsaciones por minuto
Frecuencia máxima esperada: 170,00 pulsaciones por minuto

Process finished with exit code 0
```

EJERCICIO 5: PERFIL MÉDICO

Descripción

En este ejercicio se diseñó una clase llamada PerfilMedico, la cual representa un perfil médico básico de una persona. La clase contiene atributos como primer nombre, apellido, sexo, fecha de nacimiento, altura en centímetros y peso en kilogramos.

Además de los métodos establecer y obtener, se implementaron métodos para calcular la edad, la frecuencia cardíaca máxima, el rango de frecuencia cardíaca esperada y el índice de masa corporal, conocido como BMI.

El BMI se calcula mediante la fórmula:

$$\text{BMI} = \text{peso en kilogramos} / \text{altura en metros}^2$$

También se muestra la tabla de valores del BMI:

Bajo peso: menos de 18.5

Normal: entre 18.5 y 24.9

Sobrepeso: entre 25 y 29.9

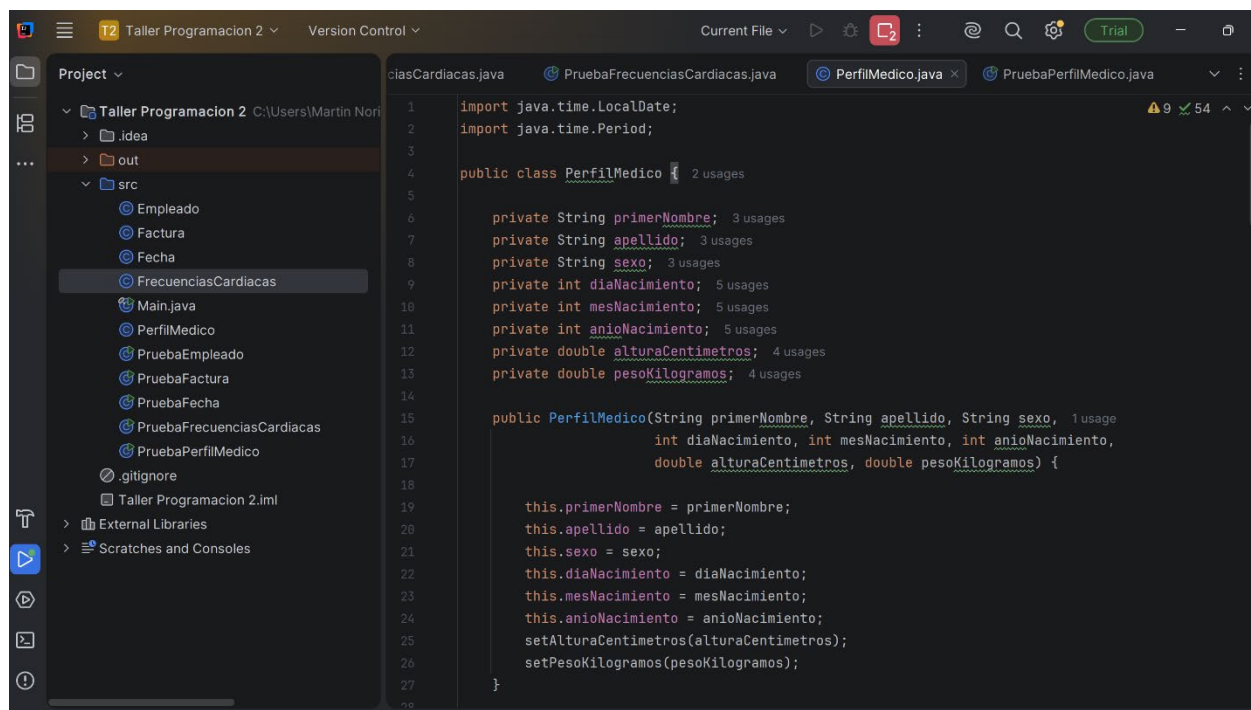
Obeso: 30 o más

Herramienta de entrada utilizada

Ventanas emergentes mediante JOptionPane.

Código fuente

Captura de PerfilMedico.java



```
1 import java.time.LocalDate;
2 import java.time.Period;
3
4 public class PerfilMedico {
5
6     private String primerNombre;
7     private String apellido;
8     private String sexo;
9     private int diaNacimiento;
10    private int mesNacimiento;
11    private int anioNacimiento;
12    private double alturaCentimetros;
13    private double pesoKilogramos;
14
15    public PerfilMedico(String primerNombre, String apellido, String sexo,
16                        int diaNacimiento, int mesNacimiento, int anioNacimiento,
17                        double alturaCentimetros, double pesoKilogramos) {
18
19        this.primerNombre = primerNombre;
20        this.apellido = apellido;
21        this.sexo = sexo;
22        this.diaNacimiento = diaNacimiento;
23        this.mesNacimiento = mesNacimiento;
24        this.anioNacimiento = anioNacimiento;
25        setAlturaCentimetros(alturaCentimetros);
26        setPesoKilogramos(pesoKilogramos);
27    }
28 }
```

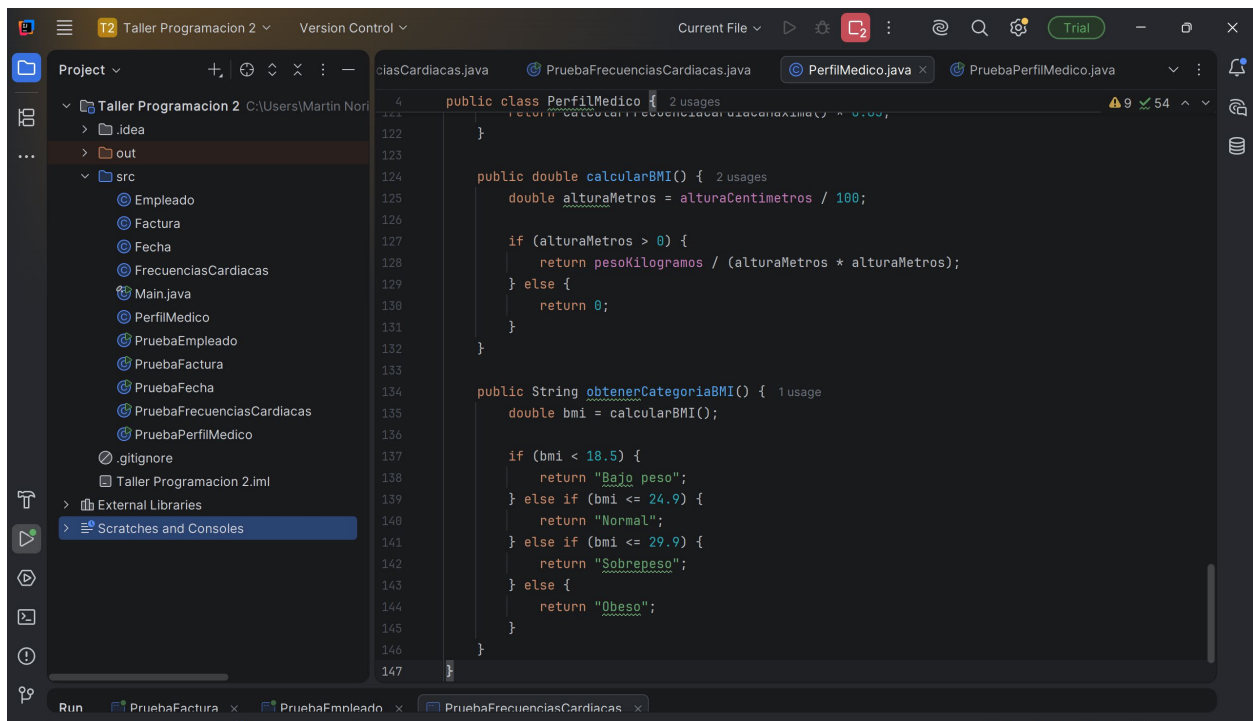


```
4 public class PerfilMedico { 2 usages
15 public PerfilMedico(String primerNombre, String apellido, String sexo, 1 usage
27 }
28
29 public void setPrimerNombre(String primerNombre) { no usages
30     this.primerNombre = primerNombre;
31 }
32
33 public String getPrimerNombre() { 1 usage
34     return primerNombre;
35 }
36
37 public void setApellido(String apellido) { no usages
38     this.apellido = apellido;
39 }
40
41 public String getApellido() { 1 usage
42     return apellido;
43 }
44
45 public void setSexo(String sexo) { no usages
46     this.sexo = sexo;
47 }
48
49 public String getSexo() { 1 usage
50     return sexo;
51 }
52 }
```

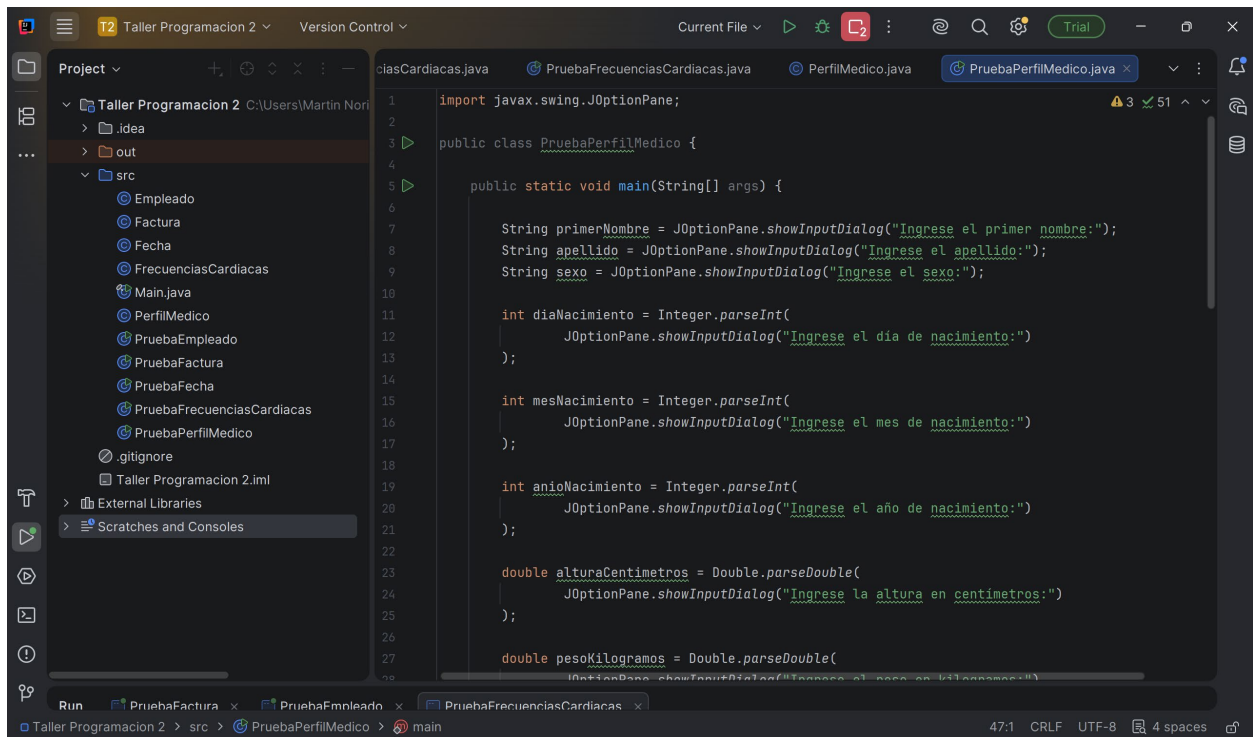
```
4 public class PerfilMedico { 2 usages
50     return sexo;
51 }
52
53 public void setDiaNacimiento(int diaNacimiento) { no usages
54     this.diaNacimiento = diaNacimiento;
55 }
56
57 public int getDiaNacimiento() { no usages
58     return diaNacimiento;
59 }
60
61 public void setMesNacimiento(int mesNacimiento) { no usages
62     this.mesNacimiento = mesNacimiento;
63 }
64
65 public int getMesNacimiento() { no usages
66     return mesNacimiento;
67 }
68
69 public void setAnioNacimiento(int anioNacimiento) { no usages
70     this.anioNacimiento = anioNacimiento;
71 }
72
73 public int getAnioNacimiento() { no usages
74     return anioNacimiento;
75 }
76 }
```

```
public class PerfilMedico {  
    public void setAlturaCentimetros(double alturaCentimetros) {  
        if (alturaCentimetros > 0) {  
            this.alturaCentimetros = alturaCentimetros;  
        } else {  
            this.alturaCentimetros = 0;  
        }  
    }  
  
    public double getAlturaCentimetros() {  
        return alturaCentimetros;  
    }  
  
    public void setPesoKilogramos(double pesoKilogramos) {  
        if (pesoKilogramos > 0) {  
            this.pesoKilogramos = pesoKilogramos;  
        } else {  
            this.pesoKilogramos = 0;  
        }  
    }  
  
    public double getPesoKilogramos() {  
        return pesoKilogramos;  
    }  
  
    public String mostrarFechaNacimiento() {  
        return diaNacimiento + "/" + mesNacimiento + "/" + anioNacimiento;  
    }  
}
```

```
    public String mostrarFechaNacimiento() {  
        return diaNacimiento + "/" + mesNacimiento + "/" + anioNacimiento;  
    }  
  
    public int calcularEdad() {  
        LocalDate fechaNacimiento = LocalDate.of(anioNacimiento, mesNacimiento, diaNacimiento);  
        LocalDate fechaActual = LocalDate.now();  
        return Period.between(fechaNacimiento, fechaActual).getYears();  
    }  
  
    public int calcularFrecuenciaCardiacaMaxima() {  
        return 220 - calcularEdad();  
    }  
  
    public double calcularFrecuenciaCardiacaEsperadaMinima() {  
        return calcularFrecuenciaCardiacaMaxima() * 0.50;  
    }  
  
    public double calcularFrecuenciaCardiacaEsperadaMaxima() {  
        return calcularFrecuenciaCardiacaMaxima() * 0.85;  
    }  
}
```



Captura de PruebaPerfilMedico.java



```
3 public class PruebaPerfilMedico {
5     public static void main(String[] args) {
27         double pesoKilogramos = Double.parseDouble(
28             JOptionPane.showInputDialog("Ingrese el peso en kilogramos:")
29         );
30
31         PerfilMedico persona = new PerfilMedico(
32             primerNombre,
33             apellido,
34             sexo,
35             diaNacimiento,
36             mesNacimiento,
37             anioNacimiento,
38             alturaCentimetros,
39             pesoKilogramos
40         );
41
42         String resultado = "PERFIL MÉDICO\n\n"
43             + "Nombre completo: " + persona.getPrimerNombre() + " " + persona.getApellido() + "\n"
44             + "Sexo: " + persona.getSexo() + "\n"
45             + "Fecha de nacimiento: " + persona.mostrarFechaNacimiento() + "\n"
46             + "Altura: " + persona.getAlturaCentimetros() + " cm\n"
47             + "Peso: " + persona.getPesoKilogramos() + " kg\n\n"
48
49             + "Edad: " + persona.calcularEdad() + " años\n"
50             + "BMI: " + String.format("%.2f", persona.calcularBMI()) + "\n"
51             + "Categoría BMI: " + persona.obtenerCategoriaBMI() + "\n\n"
52     }
```

```
51         + "Categoría BMI: " + persona.obtenerCategoriaBMI() + "\n\n"
52
53         + "Frecuencia cardíaca máxima: "
54         + persona.calcularFrecuenciaCardiacaMaxima()
55         + " pulsaciones por minuto\n"
56
57         + "Rango de frecuencia cardíaca esperada:\n"
58         + "Mínima 50%: "
59         + String.format("%.2f", persona.calcularFrecuenciaCardiacaEsperadaMinima())
60         + " pulsaciones por minuto\n"
61
62         + "Máxima 85%: "
63         + String.format("%.2f", persona.calcularFrecuenciaCardiacaEsperadaMaxima())
64         + " pulsaciones por minuto\n"
65
66         + "VALORES DE BMI\n"
67         + "Bajo peso: menos de 18.5\n"
68         + "Normal: entre 18.5 y 24.9\n"
69         + "Sobrepeso: entre 25 y 29.9\n"
70         + "Obeso: 30 o más";
71
72         JOptionPane.showMessageDialog(parentComponent: null, resultado);
73     }
74 }
```

Caso de prueba

Primer nombre: Carlos

Apellido: Pérez

Sexo: Masculino

Día de nacimiento: 10

Mes de nacimiento: 5

Año de nacimiento: 2005

Altura: 170 cm

Peso: 70 kg

Resultado esperado

Edad: depende de la fecha actual

BMI aproximado: 24.22

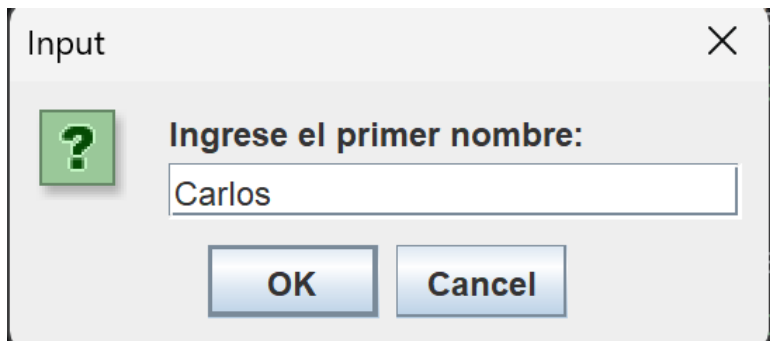
Categoría BMI: Normal

Frecuencia cardíaca máxima: $220 - \text{edad}$

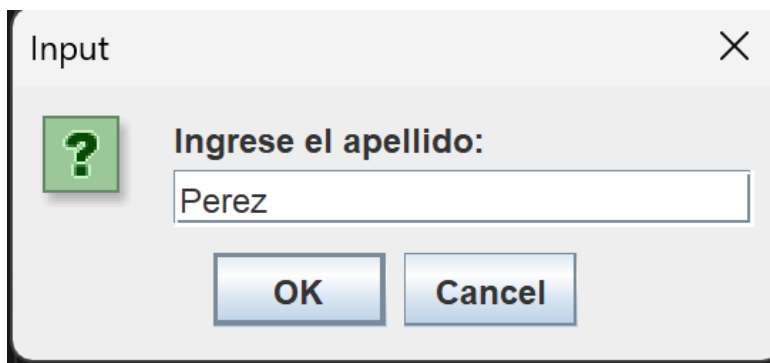
Rango de frecuencia cardíaca esperada: entre 50% y 85%

Evidencia de ejecución

Captura del ingreso de datos



The screenshot shows a standard Windows-style dialog box titled 'Input' with a close button (X) in the top right corner. On the left side, there is a green square icon containing a white question mark. To the right of this icon, the text 'Ingrese el primer nombre:' is displayed. Below this text is a text input field containing the word 'Carlos'. At the bottom of the dialog, there are two buttons: 'OK' and 'Cancel'.



The screenshot shows a second 'Input' dialog box, identical in layout to the first one. It features a green square icon with a white question mark on the left. The text 'Ingrese el apellido:' is positioned to the right of the icon. Below this text is a text input field containing the word 'Perez'. At the bottom, there are 'OK' and 'Cancel' buttons.

Input ✕

 **Ingrese el sexo:**

Masculino

OK Cancel

Input ✕

 **Ingrese el día de nacimiento:**

10

OK Cancel

Input ✕

 **Ingrese el mes de nacimiento:**

5

OK Cancel

Input ✕

 **Ingrese el año de nacimiento:**

2005

OK Cancel

Input

 Ingrese la altura en centímetros:

OK Cancel

Input

 Ingrese el peso en kilogramos:

OK Cancel

Captura del resultado final del perfil médico

Project: Taller Programacion 2

- src
 - Empleado
 - Factura
 - Fecha
 - FrecuenciasCardiacas
 - Main.java
 - PerfilMedico
 - PruebaEmpleado
 - PruebaFactura
 - PruebaFecha
 - PruebaFrecuenciasCardiacas
 - PruebaPerfilMedico

Message

PERFIL MÉDICO

Nombre completo: Carlos Perez
Sexo: Masculino
Fecha de nacimiento: 10/5/2005
Altura: 170.0 cm
Peso: 70.0 kg

Edad: 20 años
BMI: 24,22
Categoría BMI: Normal

Frecuencia cardiaca máxima: 200 pulsaciones por minuto
Rango de frecuencia cardiaca esperada:
Mínima 50%: 100,00 pulsaciones por minuto
Máxima 85%: 170,00 pulsaciones por minuto

VALORES DE BMI
Bajo peso: menos de 18.5
Normal: entre 18.5 y 24.9
Sobrepeso: entre 25 y 29.9
Obeso: 30 o más

OK

REPOSITORIO DE GITHUB

Link del repositorio:

<https://github.com/MartinAimacana/S4-Taller-de-clases-objetos-y-m-todos-Martin-Aimaca-a-Alex-Palma.git>

CONCLUSIÓN

En este taller se aplicaron los conocimientos adquiridos sobre clases y objetos en Java. Se desarrollaron diferentes ejercicios utilizando atributos privados, constructores, métodos establecer y obtener, y métodos de cálculo según los requerimientos de cada problema.

También se implementaron clases de prueba para demostrar el funcionamiento de cada clase creada. A través de los casos de prueba se verificó que los programas realizan correctamente los cálculos solicitados, como el monto de una factura, salario anual, aumento de sueldo, formato de fecha, frecuencia cardíaca esperada y cálculo del BMI.

Finalmente, se utilizaron diferentes mecanismos de entrada de datos, como ventanas emergentes, Scanner y BufferedReader, cumpliendo con las indicaciones establecidas en la actividad.